Course: ENSF 694

Final Project

Instructor: Maan Khedr

Student Names: Eric Godin, Dylan Bowersock, Zohara Kamal

Submission Date: 08 18 2026

1: Introduction

This project, and the corresponding repository, addresses the problems outlined in "ENSF694 Term Project Guidelines.pdf" found within the repository. These problems are as followed:

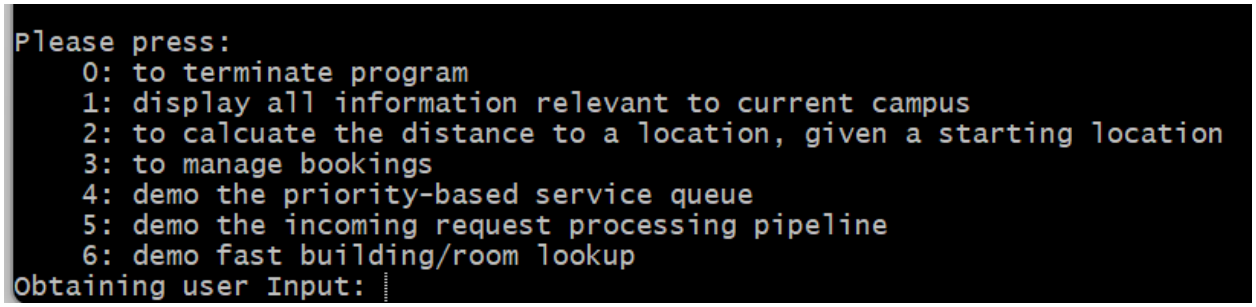
1. Create and navigate (via a shortest distance algorithm) a Campus environment
2. Allow and manage the booking of rooms within the Campus
3. Create and process requests that simulate a service Queue and priority Queue
4. Maintain fast lookup and balancing in all aspects

The team has addressed these problems via all of the cpp & h files within the repository.

1. CNEMS.cpp is functionally, the entry point into the program and the Client Interface
2. Building.cpp/h, Campus.cpp/h, Hashtable.cpp/h, Room.cpp/h, Request.cpp/h, booking.cpp/h, AVL.cpp/h, HashTable.h, and graph.cpp/h are all custom created classes/objects that programmatically solve these issues
3. All text files, including RoomBooking.txt, RoomInformation.txt, and CampusMap.txt, serve as input files to load the information required for the program to run. eg) CampusMap.txt is a list of all the campus buildings and its corresponding pathways.

Instructions to run the program:

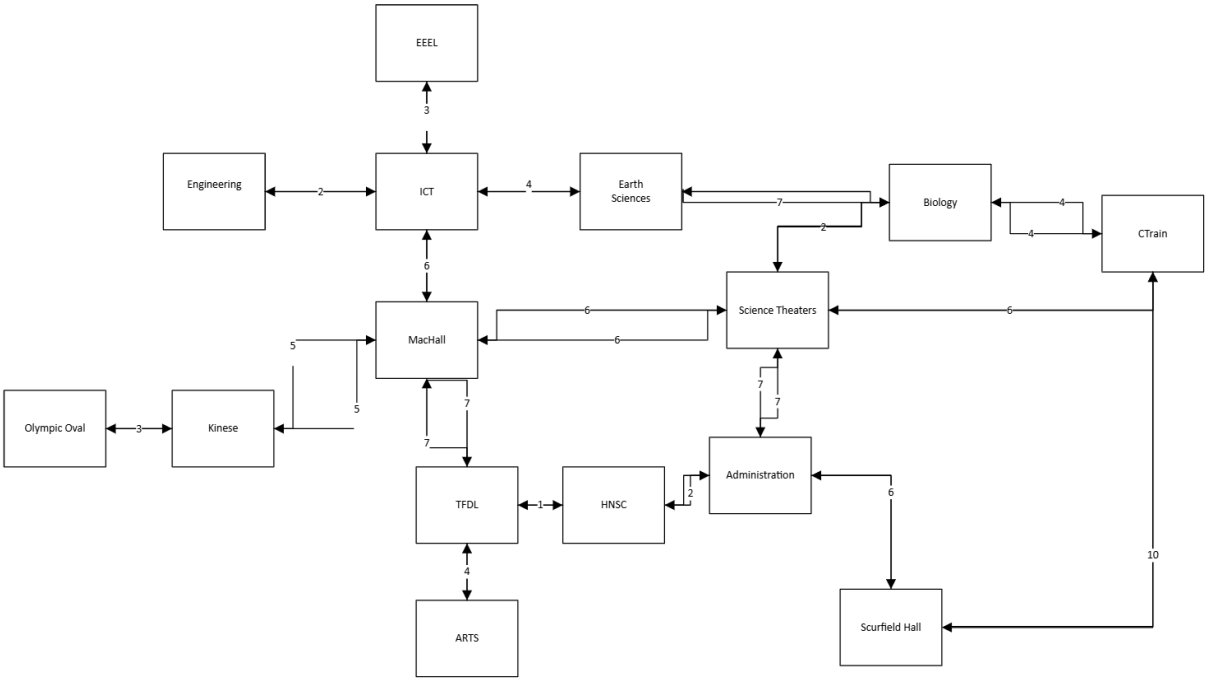
- To start the program, please run the "final.exe" file found in the repository.
- The entry point to the program will be a menu with several options. These options are directly related to the project problems and desired outcomes.
- Navigation is only integer input (for the different text options) and input of strings specific to buildings and rooms. NOTE, input other than what is expected may crash the program; it was assumed the user interacts with the program as intended.



Campus Layout

The below image is a visual representation of the campus layout used for this project. There is a total of 15 buildings and 36 pathways. Note that from each building, visually it looks like there is

only 1 path way but, from an implementation perspective, there are two; one going and one returning. The building count and pathways are calculated computationally in the program.



2: Design decisions

2.1: Campus Map and Shortest Path Navigation

The Campus Map is … the University of Calgary! This was chosen, of course, because it's the perfect representation of our problem scenario and 'close to home'. 15 buildings were chosen at random, gave various pathways based on intuitive sense, and assigned various weights based on how long, roughly, it takes to walk from each building.

The algorithm for the shortest path was chosen to be Dijkstra's algorithm. This algorithm was chosen, partly because of familiarity, but also because of efficacy. By utilizing a 'greedy' approach to node selection we can quickly search and compute our shortest distance on the campus. This approach requires a non-negative weight (or distance/time) but given we are calculating lengths amongst a campus pathway, this is a non issue. Dijkstar's algorithm intrinsically is a tree data structure as it only visits a node once (from a parent mode) and contains no cycles. This was chosen over other data structures, like a breath-first search, because the shortest path algorithm, in this specific situation, required it to be weighted and non-negative.

2.2: Route History and Navigation Undo

The route history is stored in 2 vectors, composed of building pointers, that function as a log of the past queries. One vector logs the starting building and the other logs the end building. These vectors work in parallel to one another. When a user requests, and completes, a campus shortest path calculation, their request for the start and end building is logged in these vectors. The vectors function as a stack. If the user requests for the past query, this request, and the associated buildings, are popped from the vector. A vector was chosen for three reasons: they are easy to implement, they can grow in size, and due to the limited queries we will be expecting, are not a hindrance to performance and memory. An alternative could have been to create a stack on our own, with a fixed size.

The navigation undo however, where a user ‘walks back’ to the previous building, is implemented via a linked list and created when the Dijkstra algorithm runs (during the shortest path calculation). Each building object stores the previous building (or node) along it’s shortest path. By ‘walking back’ along the linked list, you functionally implement a navigation undo. A linked list was chosen because it’s easy to implement, and performance mindful, when you are already running Dijkstra’s algorithm. An alternative to this strategy would have been to implement a ‘log’ of locations visited, either via a vector or a stack. However, compared to a linked list, this would have had a cost with respect to both memory and performance

2.3: Room and Event Booking System

For the event booking system, we chose to use an AVL tree to store the bookings. Each room has its own AVL tree, and each node in the tree contains a Booking object that holds the booking year, month, day, and starting time, as well as its duration and the purpose of the booking (IE “class”, “event”). The AVL tree was chosen for two reasons; fast insertion/deletion/lookup to meet the bonus section 2.7 requirements, and also because it allows for relatively simple functions to print all bookings in order.

The choice to have each room have its own AVL tree was for our use case, we want to separate bookings per room for display purposes. As well, the tree was chosen to be sorted by the time the bookings begin, converted to an integer long so that each booking was stored in chronological order.

We considered using a simple stack or linked list for the bookings, either using an array to store instances of Booking objects or simply linking each node one after the other. While this would be easy to implement at first, it would be more difficult to perform searches and it was not computationally efficient at all. The AVL tree was the better choice for performance.

We also considered, instead of designing a Booking object, just holding a linked list inside each Room that contained an array of arrays containing the information for each booking. In the end, a Booking object was much cleaner to implement.

2.4: Priority-Based Service Queue

The complexity of adding a request to the priority queue is O(n), where n is the number of requests already waiting, since addRequest() has to walk the sorted linked list from the front until it finds the correct position for the new request based on priority and, for ties, arrival order. Although a binary heap could achieve this insertion in O(log n), our implementation intentionally uses a simpler sorted list because the queue is read (via serveNext()) far more often than it is written to in a service-desk scenario, and with only three priority levels, the difference between O(n) and O(log n) insertion is negligible in practice. Serving the next request, by contrast, is O(1) in the best, average, and worst case, because the highest-priority, earliest-arrived request is guaranteed to already be sitting at the front of the list by construction, so no search is ever required. The space complexity is O(n), where n is the number of requests currently queued, since the queue is a linked list with exactly one node allocated per request and no additional overhead structures.

2.5: Fast Building and Resource Lookup

The complexity of the fast lookup is O(1) on average and in the best case for insert, remove, and lookup, since each operation computes a single hash of the key and then only has to scan the short chain of entries stored in that bucket. In the worst case if many distinct keys happened to hash into the same bucket, this could degrade to O(n), where n is the number of stored entries; however, this scenario is actively avoided in our implementation, since the table monitors its load factor and automatically doubles its bucket count and rehashes every existing entry once the load factor exceeds 0.75 which keeps the average chain length, and therefore the average lookup time, effectively constant no matter how many buildings or rooms are added. The space complexity is O(n) for the n stored building and room records, plus O(b) for the bucket array itself, where b grows roughly proportionally to n as the table rehashes, so total space remains linear in the number of stored records.

2.6: Incoming Request Processing

The complexity of the request pipeline is O(1) in the best, average and worst cases for both enqueuing and dequeuing a request, since the pipeline maintains explicit front and back pointers and every operation is a constant number of pointer reassignments; there is no scenario in which a request needs to be searched for or shifted, unlike an array-based queue where dequeuing from the front would require shifting every remaining element. The space complexity is O(n) where n is the number of requests currently pending in the pipeline, since the structure is a linked list with exactly one node per request and no auxiliary indexing structures.

2.7 (Bonus) Balanced Event Index

The choice was made to use an AVL tree for the event index. This is because, compared to other structures like linked lists and regular binary search trees, AVL trees have faster lookup that meets the bonus requirements, completing lookup and insertions in O(log n) time. This is a feature of AVL trees that holds true even in adversarial insertion patterns, thanks to the tree's balancing.

For the AVL tree, each node contains a booking object (with date, time, duration, and purpose of booking), as well as a *starting time* and *ending time* for the booking. These times are calculated just for the AVL tree and are used to search the tree as well as detect potential time conflicts when adding bookings

The key for each node is the *starting time* of the booking. This allows for searching chronologically, and presenting the results in chronological order within O(log n) time.

After each insertion or deletion, the tree is rebalanced according to the balance factors of the nodes.

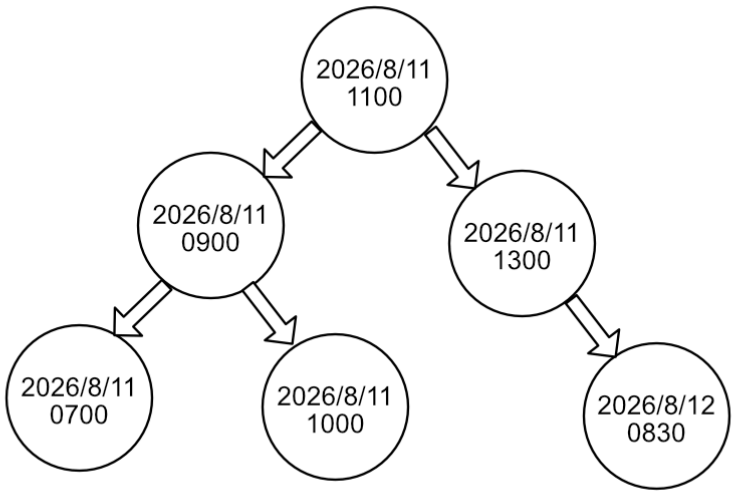
An example of node insertion and tree rebalancing is shown here:
Before adding a new node, here are the bookings for ICT 204:

```
For room ict_204 the bookings are as shown:

Booking time: 2026/8/11 at 700
Duration: 110 minutes
Purpose: Class

Booking time: 2026/8/11 at 900
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/11 at 1000
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/11 at 1100
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/11 at 1300
Duration: 110 minutes
Purpose: Class

Booking time: 2026/8/12 at 830
Duration: 50 minutes
Purpose: Event
```

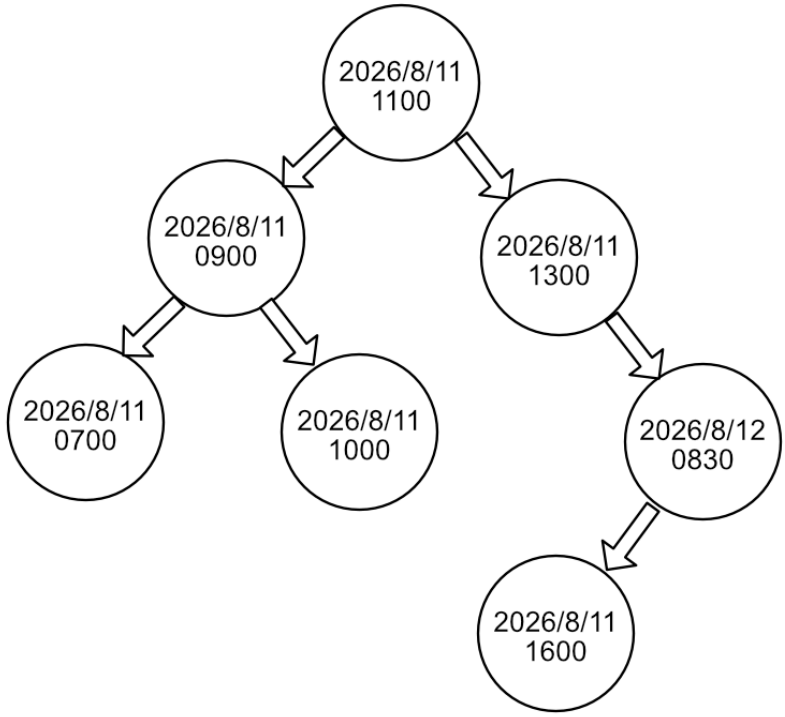
After adding a new node, here are the bookings:

```
For room ict_204 the bookings are as shown:

Booking time: 2026/8/11 at 700
Duration: 110 minutes
Purpose: Class

Booking time: 2026/8/11 at 900
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/11 at 1000
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/11 at 1100
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/11 at 1300
Duration: 110 minutes
Purpose: Class

Booking time: 2026/8/11 at 1600
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/12 at 830
Duration: 50 minutes
Purpose: Event
```

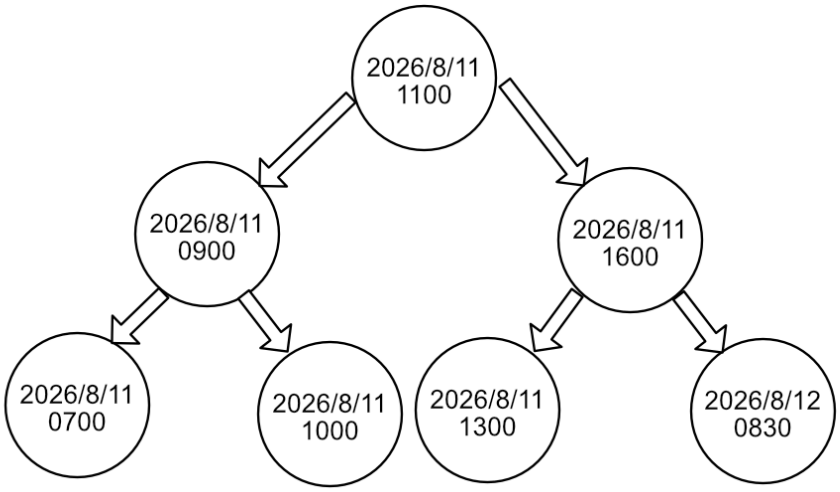
The tree before insertion:



The tree during insertion, before rotation:



The tree after insertion and rotation:



As you can see, the tree remains in order even after adding a new node.

3: Complexity Analysis

The complexity of Djikstra's algorithm, which is used to calculate the shortest path, is O(n2) in the worst case. Although Dijkstra's algorithm is typically faster operating at O(n), and reflected in this code as such, this instantiation of Dijkstra's also include a vector of visited locations that is checked upon every node; therefore making the algorithm run, on the whole, as O(n2). The average case is O(n2) and the best case would be O(n); the best case is achieved when the query for visited is either consulted very little if at all (the algorithm proceeds with the Queue linearly).

The complexity of the route navigation and last query is O(1). Both operations, a linked list and a vector respectively, only require one iteration at most to achieve it's ends.

For the AVL tree, it is well established that complexity for searches is O(log n). This is the case for our AVL tree searches, insertions, and deletions, and by proxy the booking system operations have O(log n) complexity.

In a worst case for the AVL tree, that is inserting a node that goes on the very end of a leftmost or rightmost subtree that requires rotations, even these cases do not need to go along the entire tree, only their branches. Because each branch is balanced, operations on the AVL tree do not need to visit every node which is the cause of its lower complexity.

The space complexity of an AVL tree is O(n). Each node requires enough space for itself, and is not duplicated.

4: Challenges and Lessons Learned

Dylan
I found two things very challenging in this project: continuous expansion of a project and the implementation of superior data structures. On the former, I learned that it's very important to start out with a solid road map, is scalable, and the proper data structures to begin with. As your program grows in size, if not done properly, I find myself 'tacking' on modules that were costly relative to performance, memory, or complexity (or all 3), simply because the foundation of the program ultimately dictated it; it would have required an overhaul to do it any other way. On th latter, I leared, through this project, that there is a very clear difference between 'it runs' and 'it runs well'. A linked list, a stack, and a vector can all achieve the same end but it's very important to know the tradeoffs of each, and the advantages, if you want to optimize your project.

Eric
The largest challenges I had lay in implementing the AVL tree. Many of the functions in the tree are recursive and keeping the tree straight in my head while programming was quite difficult. I

have dozens of scratchpad pages full of AVL trees with nodes circled and crossed out and arrows pointing every which way. At the start, I thought I could get away with containing everything in just the room.h and room.cpp files; but I realized creating an AVL.h and AVL.cpp would make things cleaner. Deletions gave me a hard time and I credit my team members for helping me catch bugs in my code, a bug that gave me a hard time turned out to be a set of missing {} causing a line of code to be executed every iteration instead of only within an “if ()”. A lesson learned is to keep your code as neat and commented as possible, and not to wait to the end to comment. If the AVL code was messy I would have gotten completely lost since it’s so hard to visualize large trees with recursion. I ended up having to create more functions than expected, many of which are called a single time or only used to support one other function. It’s important to recognize what functions you need even just for your own clarity while programming. Sometimes, it makes sense to implement restrictions on user input as well. Originally we allowed bookings to be created using fractional hours, like 7.5 standing in for 7:30, but that would complicate searching because we’d be trying to find a float == float. So I changed it to only accept 24 hour time, 0730, and wrote a function to convert that to an integer. Datatypes are important, and spotting an issue before it happens like float imprecision is very important.

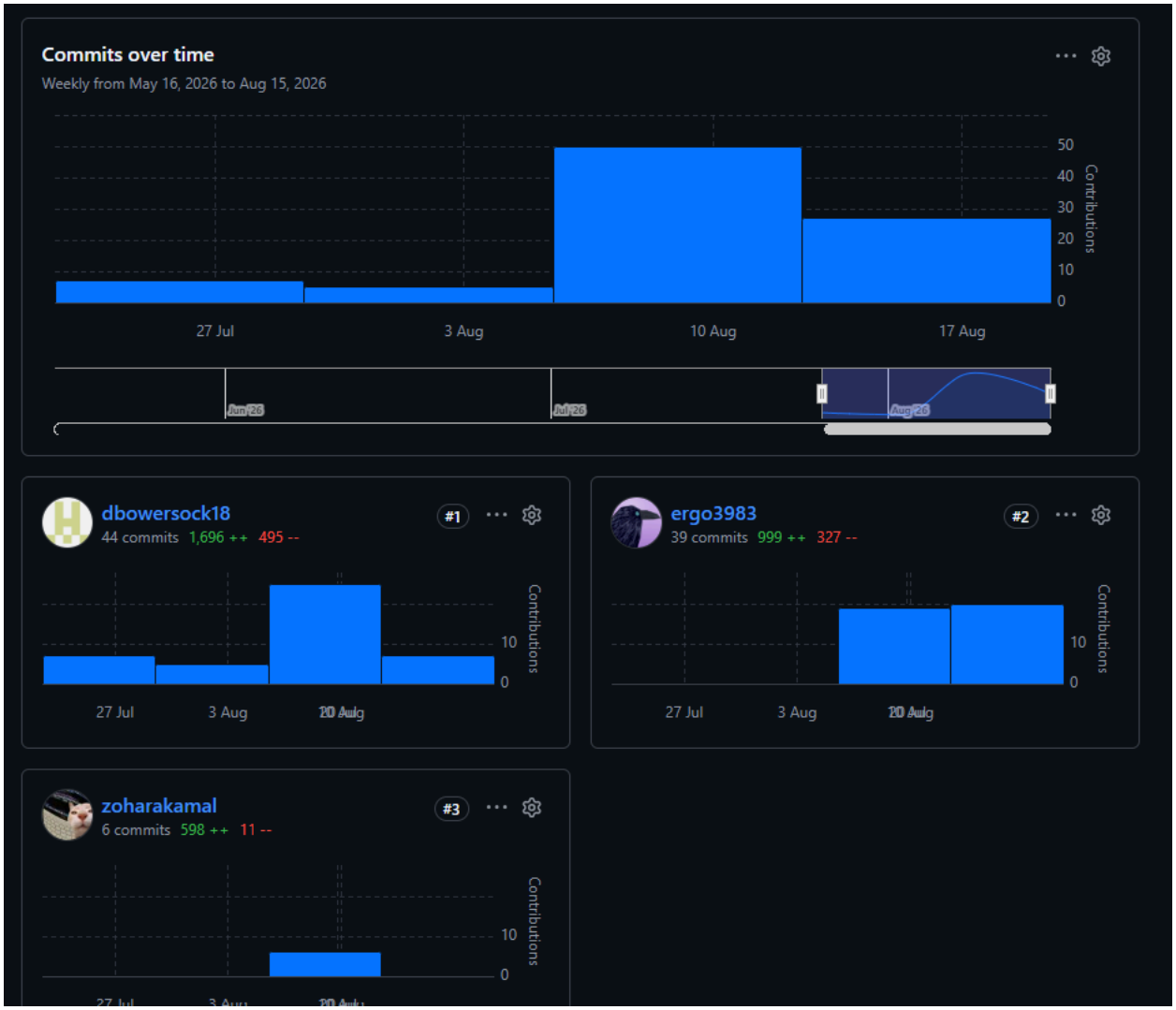
Zohara
My biggest challenge was with the hash table for 2.5. I initially tried to split it into a HashTable.h and HashTable.cpp the same way we structured every other class in the project but since HashTable is a template (HashTable<Building>, HashTable<Room>), the compiler couldn't find the function definitions at link time unless they lived entirely in the header,templates only get instantiated where they're used and a separate.cpp file isn't visible at those call sites. I ended up moving everything into HashTable.h and adding a comment explaining why so I wouldn't "fix" it back to two files by accident later. It was a good reminder that C++ templates don't follow the same compilation model as regular classes.

The priority queue for 2.4 taught me a similar lesson about edge cases. My first version of addRequest() only handled inserting in the middle or end of the list and kept breaking when a request needed to go at the very front (either because the queue was empty or because a new emergency request needed to jump ahead of everything). I had to add a separate check against front before falling into the general traversal loop. It also took me a couple of tries to get the tie-breaking right. Two requests at the same priority level need to stay in the order they arrived, so I had to sort by priority first arrival id second, not just priority alone; otherwise, a standard request could jump ahead of an older standard request. Small edge cases like that taught me to test the "insert at the boundary" case explicitly instead of just the general case.

5: Contributions

Eric	Dylan	Zohara
• Booking objects and system • AVL tree & AVL functions	• Campus Map • Shortest Path Navigation (implementation of Djiskstra's algorithm) • Query and last navigation	• Priority-based service queue • Fast building and resource lookup • Incoming request processing pipeline

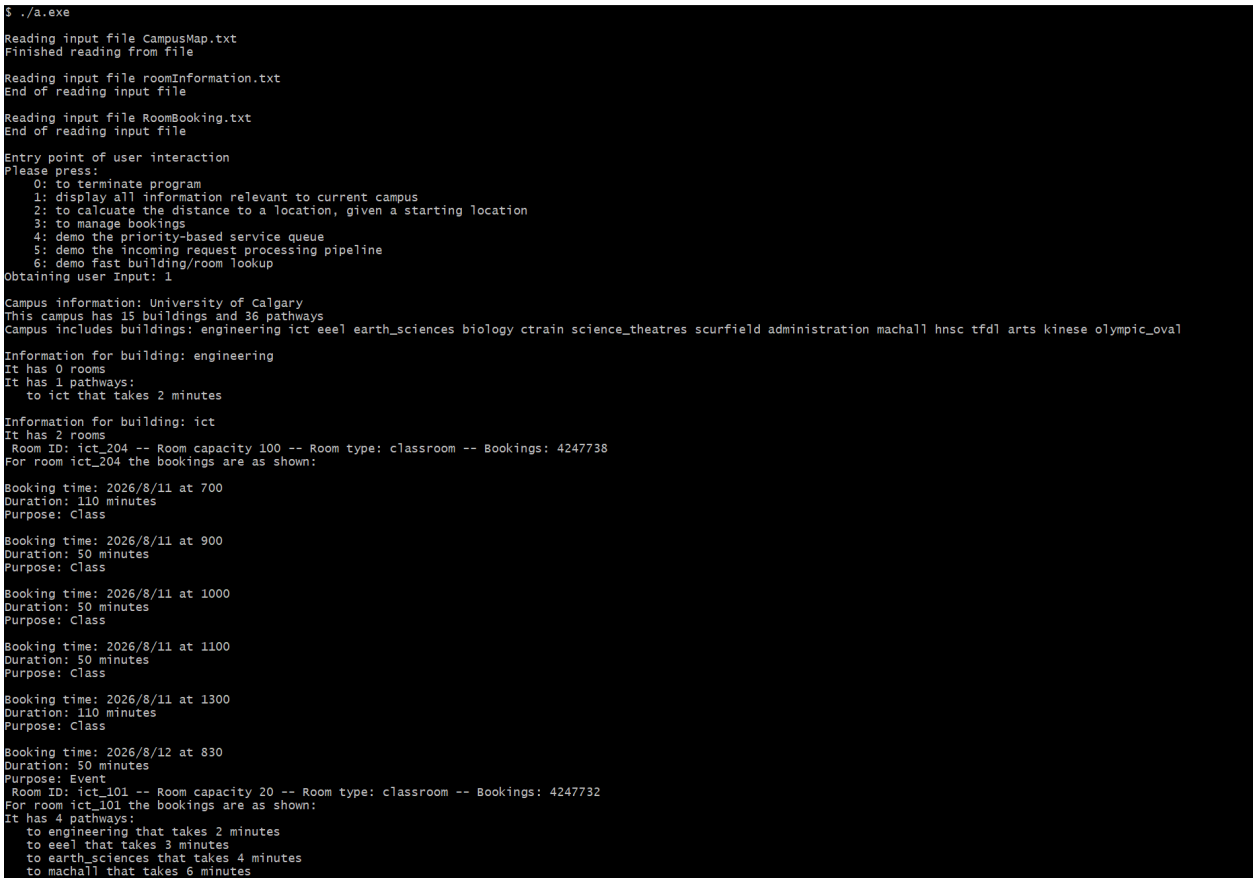
Github Contributional graph is below



6: Demos

Printing information

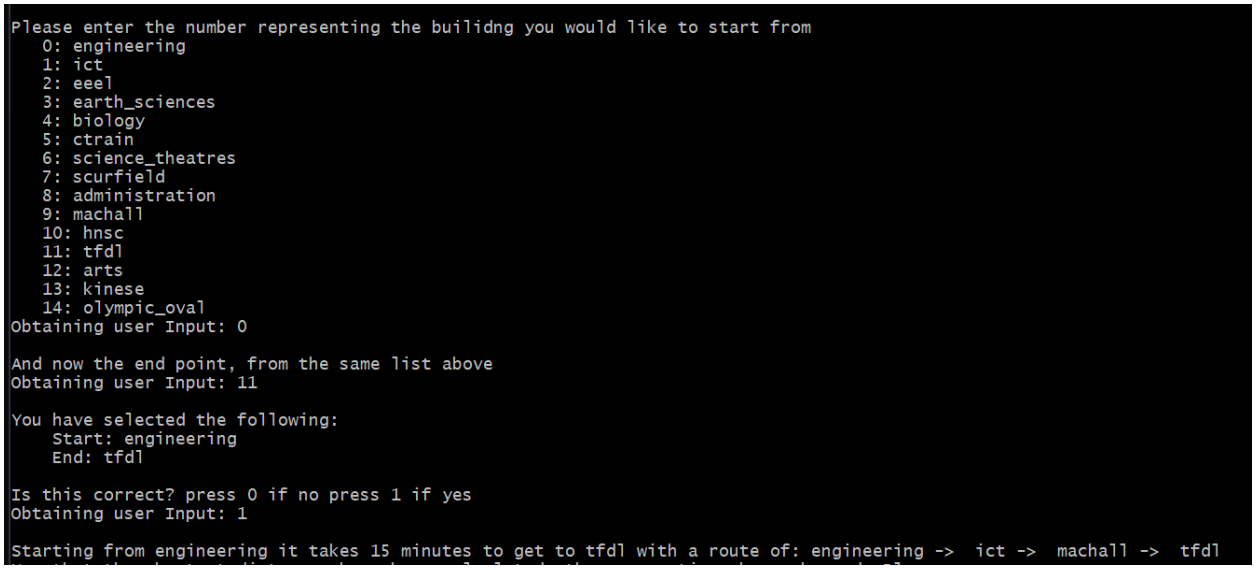
The screenshot below demonstrates the output when the user selects the option to display information regarding the campus. New buildings, rooms, and bookings can be added quite easily, and at volume, using the text files.



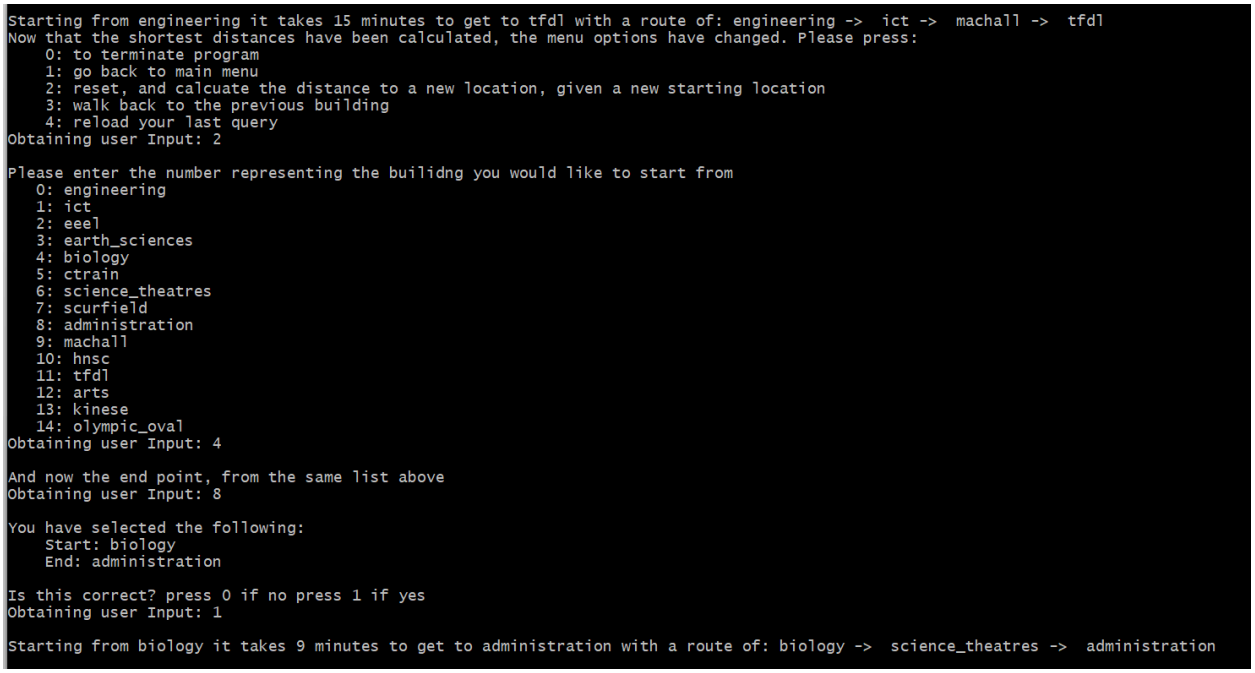
6.1: Shortest Path Query

When the user selects the shortest path query, they are prompted with a list of buildings and asked to provide the start and end location. After a quick verification the input is correct, the user is given the pathway between the start and end locations, and the distance/time to and

from. The algorithm behind this is discussed in the design decisions section below.



Resetting the query and selecting another option



6.2: Undo Navigation

If a user would like to undo the path they can either: Walk back to the previous building

```
Starting from biology it takes 9 minutes to get to administration with a route of: biology ->  science_theatres ->  administration

Now that the shortest distances have been calculated, the menu options have changed. Please press:
    0: to terminate program
    1: go back to main menu
    2: reset, and calcuate the distance to a new location, given a new starting location
    3: walk back to the previous building
    4: reload your last query
Obtaining user Input: 3

Now ... starting from biology it takes 2 minutes to get to science_theatres with a route of: biology ->  science_theatres

Now that the shortest distances have been calculated, the menu options have changed. Please press:
    0: to terminate program
    1: go back to main menu
    2: reset, and calcuate the distance to a new location, given a new starting location
    3: walk back to the previous building
    4: reload your last query
Obtaining user Input:
```

Or, reload the last query completely

```
Now ... starting from biology it takes 2 minutes to get to science_theatres with a route of: biology ->  science_theatres

Now that the shortest distances have been calculated, the menu options have changed. Please press:
    0: to terminate program
    1: go back to main menu
    2: reset, and calcuate the distance to a new location, given a new starting location
    3: walk back to the previous building
    4: reload your last query
Obtaining user Input: 4


The previous query you have requested is:
    Start: engineering
    End: tfdl

Starting from engineering it takes 15 minutes to get to tfdl with a route of: engineering ->  ict ->  machall ->  tfdl

Now that the shortest distances have been calculated, the menu options have changed. Please press:
    0: to terminate program
    1: go back to main menu
    2: reset, and calcuate the distance to a new location, given a new starting location
    3: walk back to the previous building
    4: reload your last query
Obtaining user Input:
```

6.3: Booking Range Query

Going back to the main menu, the user has other options as well. One of them is to manage bookings; specifically to look up when a building has a booking!

```
Please press:
    0: to terminate program
    1: display all information relevant to current campus
    2: to calcuate the distance to a location, given a starting location
    3: to manage bookings
    4: demo the priority-based service queue
    5: demo the incoming request processing pipeline
    6: demo fast building/room lookup
Obtaining user Input: 3

Would you like to:
   0: Return to the main menu
   1: View all bookings for a given room
   2: View bookings for a given room, within a specified window
   3: Add booking
   4: Remove booking
Please enter a value: 2
Please type the building, followed by the room id. eg) ict ict_204 Press 0 for either input to return: ict ict_204

Room found!
Please enter the following, in this specific format: year day month time
For example August 11th 2026 7:30am should be entered as, exactly: 2026 8 11 0730
noting that the time is in 24 hour time

With that, please enter your date to start: 2026 8 11 830

Now please enter the date to end querying, in the same format: 2026 8 11 1100

You have entered: Booking time: 2026/8/11 at 830
 to Booking time: 2026/8/11 at 1100

Is this correct? 0 if not, 1 if yes 1

The following bookings fall within the specified window:

Booking time: 2026/8/11 at 900
Duration: 50 minutes
Purpose: Class

Booking time: 2026/8/11 at 1000
Duration: 50 minutes
Purpose: Class
```

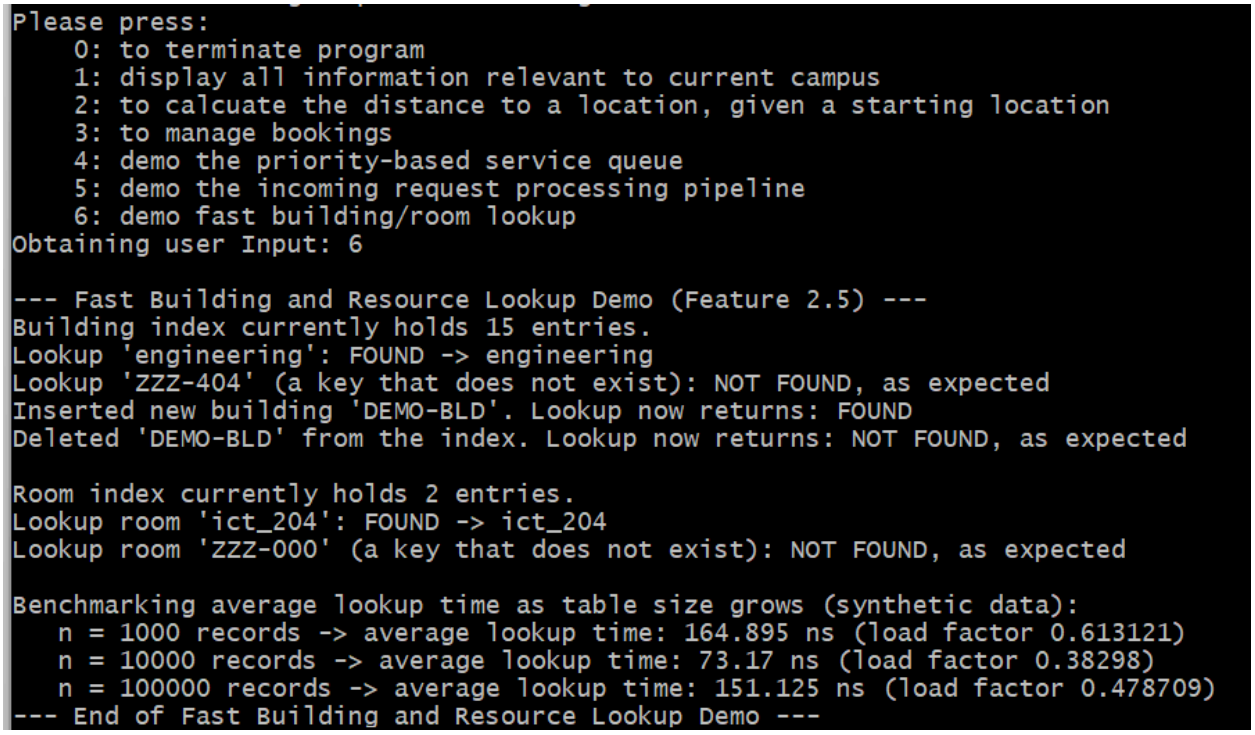
6.4: Priority Queue Demo

An important component of a campus is being able to address different requests, with different priority levels. An example of this program accomplishing that is below.

```
Please press:
    0: to terminate program
    1: display all information relevant to current campus
    2: to calcuate the distance to a location, given a starting location
    3: to manage bookings
    4: demo the priority-based service queue
    5: demo the incoming request processing pipeline
    6: demo fast building/room lookup
Obtaining user Input: 4

--- Priority-Based Service Queue Demo (Feature 2.4) ---
Adding 7 requests in arrival order:
   Arrived: [Low] Request #1: Room maintenance - flickering lights in ICT-121
   Arrived: [Standard] Request #2: IT support - projector not powering on
   Arrived: [Emergency] Request #3: Help desk - fire alarm malfunction in ENG Block
   Arrived: [Standard] Request #4: IT support - wifi outage in Library
   Arrived: [Emergency] Request #5: Help desk - gas leak reported in Science A
   Arrived: [Low] Request #6: Room maintenance - broken chair in Student Union
   Arrived: [Standard] Request #7: IT support - lab computer will not boot

Current queue, ordered by the sequence requests will be served:
   [Emergency] Request #3: Help desk - fire alarm malfunction in ENG Block
   [Emergency] Request #5: Help desk - gas leak reported in Science A
   [Standard] Request #2: IT support - projector not powering on
   [Standard] Request #4: IT support - wifi outage in Library
   [Standard] Request #7: IT support - lab computer will not boot
   [Low] Request #1: Room maintenance - flickering lights in ICT-121
   [Low] Request #6: Room maintenance - broken chair in Student Union

Serving requests:
   Now serving [Emergency] Request #3: Help desk - fire alarm malfunction in ENG Block
   Now serving [Emergency] Request #5: Help desk - gas leak reported in Science A
   Now serving [Standard] Request #2: IT support - projector not powering on
   Now serving [Standard] Request #4: IT support - wifi outage in Library
   Now serving [Standard] Request #7: IT support - lab computer will not boot
   Now serving [Low] Request #1: Room maintenance - flickering lights in ICT-121
   Now serving [Low] Request #6: Room maintenance - broken chair in Student Union
--- End of Priority-Based Service Queue Demo ---
```

6.5: Fast Lookup Demo

Another important component of this program is being able to obtain information quickly and efficiently. The screenshot below demonstrates such.



6.6: Request Pipeline Demo

Finally, the screenshot below demonstrates at least 10 requests being enqueued and dequed in arrival order.

```
--- End of Priority-Based Service Queue Demo ---
Please press:
    0: to terminate program
    1: display all information relevant to current campus
    2: to calcuate the distance to a location, given a starting location
    3: to manage bookings
    4: demo the priority-based service queue
    5: demo the incoming request processing pipeline
    6: demo fast building/room lookup
Obtaining user Input: 5

--- Incoming Request Processing Demo (Feature 2.6) ---
Enqueuing 20 sequential requests as they arrive:
   Enqueued: Request #1: Navigation query #1
   Enqueued: Request #2: Service ticket #2
   Enqueued: Request #3: Navigation query #3
   Enqueued: Request #4: Service ticket #4
   Enqueued: Request #5: Navigation query #5
   Enqueued: Request #6: Service ticket #6
   Enqueued: Request #7: Navigation query #7
   Enqueued: Request #8: Service ticket #8
   Enqueued: Request #9: Navigation query #9
   Enqueued: Request #10: Service ticket #10
   Enqueued: Request #11: Navigation query #11
   Enqueued: Request #12: Service ticket #12
   Enqueued: Request #13: Navigation query #13
   Enqueued: Request #14: Service ticket #14
   Enqueued: Request #15: Navigation query #15
   Enqueued: Request #16: Service ticket #16
   Enqueued: Request #17: Navigation query #17
   Enqueued: Request #18: Service ticket #18
   Enqueued: Request #19: Navigation query #19
   Enqueued: Request #20: Service ticket #20

Processing 20 requests, strictly in arrival order:
   Now processing Request #1: Navigation query #1
   Now processing Request #2: Service ticket #2
   Now processing Request #3: Navigation query #3
   Now processing Request #4: Service ticket #4
   Now processing Request #5: Navigation query #5
   Now processing Request #6: Service ticket #6
   Now processing Request #7: Navigation query #7
   Now processing Request #8: Service ticket #8
   Now processing Request #9: Navigation query #9
   Now processing Request #10: Service ticket #10
   Now processing Request #11: Navigation query #11
   Now processing Request #12: Service ticket #12
   Now processing Request #13: Navigation query #13
   Now processing Request #14: Service ticket #14
   Now processing Request #15: Navigation query #15
   Now processing Request #16: Service ticket #16
   Now processing Request #17: Navigation query #17
   Now processing Request #18: Service ticket #18
   Now processing Request #19: Navigation query #19
   Now processing Request #20: Service ticket #20
--- End of Incoming Request Processing Demo ---
```

AI Use Disclosure

Eric:

For finding bugs in the AVL tree code, specifically with adversarial insertion (IE inserting 90 new bookings in order), I asked Gemini to find where the bugs could be coming from. This is the conversation:

Prompt:
Help me fix a C++ issue. I'm getting a segmentation fault when adding 100 nodes to an AVL tree in ascending order. I believe the issue is either in my rotation code or my the way I'm calling the rotations. Here is the code: please find the source of the segmentation faults.

```
Node* AVLTree::rightRotate(Node* y) {
    //Right - rotates the specified node and returns the new root.
    Node *x = y->left;
    Node *z = x->right;

    x->right = y;
    y->left = z;

    if (z != nullptr){
        z->parent = x;
    }

    //Update height
    y->height = (1 + std::max(height(y->left), height(y->right)));
    x->height = (1 + std::max(height(x->left), height(x->right)));

    //Update parents
    x->parent = y->parent;
    y->parent = x;

    return x;
}

 Node* AVLTree::leftRotate(Node* x) {
    //Left - rotates the specified node and returns the new root.

    Node *y = x->right;
    Node *z = y->left;
```

```
    y->left = x;
    x->right = z;

    if (z != nullptr){
        z->parent = x;
    }

    //Update height
    y->height = (1 + std::max(height(y->left), height(y->right)));
    x->height = (1 + std::max(height(x->left), height(x->right)));

    //Update parents
    y->parent = x->parent;
    x->parent = y;

    return y;
}
```

Rotation calling:

```
if (balance < -1 && node->left != nullptr && time < node->left->data.time) {
        return rightRotate(node);
    }

    else if (balance < -1 && node->left != nullptr && time > node->left->data.time) {
       node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    else if (balance > 1 && node->right != nullptr && time > node->right->data.time) {
        return leftRotate(node);
    }

    else if (balance > 1 && node->right != nullptr && time < node->right->data.time) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
```

Gemini's response:

The segmentation fault when inserting elements in ascending order comes from two major issues in your code: **incorrect parent pointers during rotations** and **improper height updates**.

---

1. Bug: Incorrect `parent` Pointer Assignment in `z`

In both rotation functions, when `z` (the orphan subtree) is reassigned, its parent pointer is being set incorrectly.

In `rightRotate`:

```cpp
y->left = z;
if (z != nullptr) {
    z->parent = x; // ❌ WRONG: z is now y's left child, so its parent must be y!
}
```

Fix:

```cpp
if (z != nullptr) {
    z->parent = y; // ✅ Correct
}
```

In `leftRotate`:

```cpp
x->right = z;
if (z != nullptr) {
    z->parent = x; // ❌ WRONG: Here z is x's right child, but x is ALREADY its parent before,
                   // or it was y's left child. Since x becomes its parent, this happens to match,
                   // but let's double check height updates.
}
```

---

2. Bug: Updating Heights in the Wrong Order

Heights must be updated from **bottom to top** (children first, then parent).

In `rightRotate`:

`x` becomes the parent of `y`. Therefore, you must compute `y`'s height first, then `x`'s height!

```cpp
// ❌ WRONG: Computing x first uses stale height for y
x->height = (1 + std::max(height(x->left), height(x->right)));
y->height = (1 + std::max(height(y->left), height(y->right)));

// ✅ CORRECT: Order matters!
y->height = (1 + std::max(height(y->left), height(y->right)));
x->height = (1 + std::max(height(x->left), height(x->right)));

```

In `leftRotate`:

`y` becomes the parent of `x`. Therefore, `x`'s height must be updated before `y`'s!

```cpp
// ❌ WRONG in your leftRotate:
y->height = (1 + std::max(height(y->left), height(y->right)));
x->height = (1 + std::max(height(x->left), height(x->right)));

// ✅ CORRECT:
x->height = (1 + std::max(height(x->left), height(x->right)));
y->height = (1 + std::max(height(y->left), height(y->right)));

```

---

3. Bug: Incorrect Balance Factor Logic

When inserting items in **ascending order** (e.g., 1, 2, 3, 4...), the tree repeatedly becomes **right-heavy**, meaning `balance = height(left) - height(right)` will evaluate to a negative value like **`-2`**.

In standard balance factor calculations:

$$\text{balance} = \text{height}(\text{left}) - \text{height}(\text{right})$$

* **Left-heavy** ($\text{balance} > 1$): Needs Right or Left-Right rotation.

* **Right-heavy** ($\text{balance} < -1$): Needs Left or Right-Left rotation.

In your calling code, you have:

```cpp
// ❌ Checks balance < -1 for node->left (Left-heavy condition inverted!)
if (balance < -1 && node->left != nullptr && time < node->left->data.time) { ... }

```

When inserting in ascending order, `balance` becomes `-2`, but your `balance > 1` condition checks for `time > node->right->data.time`. Because of inverted condition checks or missing parent links, dangling pointers or `nullptr` dereferences occur when ascending order forces repeated `leftRotate` calls.

Fixed Code

Here are the corrected rotation functions and rotation logic:

Corrected Rotations

```cpp
Node* AVLTree::rightRotate(Node* y) {
    Node* x = y->left;
    Node* z = x->right;

    // Perform rotation
    x->right = y;
    y->left = z;

    // Update parent of z (z is now child of y)
    if (z != nullptr) {
        z->parent = y;
    }

    // Update parent links between x and y
    x->parent = y->parent;
    y->parent = x;

    // Update heights (y first, then new root x)
    y->height = 1 + std::max(height(y->left), height(y->right));
    x->height = 1 + std::max(height(x->left), height(x->right));
```

```
    return x;
}

Node* AVLTree::leftRotate(Node* x) {
    Node* y = x->right;
    Node* z = y->left;

    // Perform rotation
    y->left = x;
    x->right = z;

    // Update parent of z (z is now child of x)
    if (z != nullptr) {
        z->parent = x;
    }

    // Update parent links between x and y
    y->parent = x->parent;
    x->parent = y;

    // Update heights (x first, then new root y)
    x->height = 1 + std::max(height(x->left), height(x->right));
    y->height = 1 + std::max(height(y->left), height(y->right));

    return y;
}
```

Corrected Rotation Dispatch Logic

Assuming `balance = height(node->left) - height(node->right)`:

```cpp
// Left-Left Case (balance > 1 and inserted into left child's left)
if (balance > 1 && time < node->left->data.time) {
    return rightRotate(node);
}

// Left-Right Case (balance > 1 and inserted into left child's right)
if (balance > 1 && time > node->left->data.time) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
```

```
}

// Right-Right Case (balance < -1 and inserted into right child's right)
if (balance < -1 && time > node->right->data.time) {
    return leftRotate(node);
}

// Right-Left Case (balance < -1 and inserted into right child's left)
if (balance < -1 && time < node->right->data.time) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
}
```

This helped me debug the AVL tree in the case of adversarial insertion. Not all of the AI's advice was helpful, but it did help me find out that I was accidentally assigning the wrong parent in my RightRotate function, as well as updating the node heights in the wrong order.

Dylan:

As with Eric, I also used AI to help me debug the AVL tree and I used Co-Pilot for the purpose. The prompt, as an example, was structured as follows: "why isn't this method [copy and paste method] deleting my nodes properly". I would review the output and sometimes, not always, implement the recommended solutions if I thought they were of value.

Zohara:

Prompt:
"Help me fix a C++ bug in my priority queue. It crashes with a segfault the very first time I call addRequest() on an empty queue, and separately, when two requests have the same priority, they sometimes get served out of the order they arrived in. Here's my addRequest() function:

```
void PriorityServiceQueue::addRequest(ServiceRequest* r) {
    ListNode* newNode = new ListNode(r);
    ListNode* current = front;
    while (current->next != nullptr &&
           (current->next->element->priority < r->priority ||
            (current->next->element->priority == r->priority && current->next->element->id < r->id)))
{
```

```
        current = current->next;
    }
    newNode->next = current->next;
    current->next = newNode;
    count++;
}
```

Response:
Two separate bugs here:

1. Segfault on the first insert. The function immediately does current = front; and then reads current->next. When the queue is empty, front is nullptr, so current->next dereferences a null pointer. You need to check isEmpty() (or front == nullptr) before entering the traversal, and handle inserting at the front as its own case — the same applies any time the new request needs to become the new head (e.g., an Emergency request arriving when everything else in the queue is Standard or Low).

2. Same-priority requests served out of arrival order. Your loop condition uses current->next->element->id < r->id to decide when to stop advancing. That's backwards for stability: you want to keep walking past every existing request that has the same priority and an equal-or-earlier arrival id, so the new request lands after all of them. With strict <, the loop stops one position too early whenever an existing request has the exact same id ordering boundary, which can let a later-arriving request slot in ahead of an earlier one that shares its priority. Change the comparison to <= so ties keep advancing correctly, and the new request is only inserted once it's found a request that arrived after it.

How I used it: This matched a bug I actually had my first version segfault on the very first addRequest() call and occasionally reordered same-priority requests. I used the explanation to fix the front-insertion check and change the tie-breaking comparison from < to <=, which is what's in the final addRequest() in Request.cpp today. I wrote the original logic and the fix myself based on this explanation.